

db_bench tool

1. Write-Only Workloads

Benchmark Name	Description	Use Case	Key Parameters
fillseq	Write N values in sequential key order , async mode	Bulk loading sorted data	<code>--num</code> , <code>--key_size</code> , <code>--value_size</code>
fillseqdeterministic	Write N values in specified key order, keeps LSM tree shape deterministic	Reproducible experiments	Same as fillseq + deterministic ordering
fillrandom	Write N values in random key order , async mode	Random write throughput test	<code>--num</code> , <code>--key_size</code> , <code>--value_size</code>
filluniquerandomdeterministic	Write N values in random key order, deterministic LSM shape	Reproducible random writes	Same as fillrandom + deterministic
fillsync	Write N/100 values in random key order, sync mode	Durability-focused write test	<code>--num</code> , sync every write
fill100K	Write N/1000 values of 100KB each in random order	Large value write test	<code>--num</code> , 100KB fixed value size
overwrite	Overwrite N existing values in random key order	Update-heavy workload	<code>--num</code> , <code>--use_existing_db</code>
deleteseq	Delete N keys in sequential order	Sequential delete test	<code>--num</code>

Benchmark Name	Description	Use Case	Key Parameters
deleterandom	Delete N keys in random order	Random delete test	--num

2. Read-Only Workloads

Benchmark Name	Description	Use Case	Key Parameters
readseq	Read N times sequentially	Sequential read throughput	--num , --use_existing_db
readreverse	Read N times in reverse sequential order	Reverse scan test	--num , --use_existing_db
readrandom	Read N times in random order	Point lookup throughput	--num , --use_existing_db
readmissing	Read N missing keys in random order	Negative lookup test	--num , --use_existing_db
readtocache	1 thread reading database sequentially	Cache warmup / cold read test	Single-threaded
prefixscanrandom	Prefix scan N times in random order	Range query with prefix	--num , --prefix_size
seekrandom	N random seeks , call Next seek_nexts times per seek	Iterator / range scan test	--seek_nexts , --reverse_iterator

3. Mixed Read-Write Workloads

Benchmark Name	Description	Use Case	Key Parameters
readwhilewriting	1 writer + N threads doing random reads	Read-under-write latency	--threads , --duration
readwhilemerging	1 merger + N threads doing random reads	Read-under-merge test	--threads
readrandomwriterandom	N threads doing random-read AND random-write	Mixed workload (YCSB-A-like)	--threads , --readwritepercent
updaterandom	N threads doing read-modify-write	Update workload	--threads

Benchmark Name	Description	Use Case	Key Parameters
	for random keys	(YCSB-F-like)	
appendrandom	N threads doing read-modify-write with growing values	Append-only update test	--threads
seekrandomwhilewriting	seekrandom + 1 thread doing overwrite	Scan-under-write	--seek_nexts
seekrandomwhilemerging	seekrandom + 1 thread doing merge	Scan-under-merge	--seek_nexts

4. Merge Operator Workloads

Benchmark Name	Description	Use Case	Key Parameters
mergerandom	Same as updaterandom/appendrandom using merge operator	Merge-based updates	--merge_operator (required)
readrandommergerandom	Perform N random read-or-merge operations	Mixed read-merge workload	--merge_operator (required)

5. Transaction Workloads

Benchmark Name	Description	Use Case	Key Parameters
randomtransaction	Execute N random transactions and verify correctness	Transaction throughput	--transaction_db or --optimistic_transaction_db
randomreplacekeys	Randomly replace N keys (delete old + put new)	Key replacement test	--num

6. Time Series Workloads

Benchmark Name	Description	Use Case
<code>timeseries</code>	1 writer generates time series data + multiple readers doing random reads on id	Time series ingestion + query

7. Meta Operators (Control Flow)

Meta Operator	Description
<code>stats</code>	Print statistics (use with <code>--statistics</code> flag)
<code>compact</code>	Trigger manual compaction
<code>waitforcompaction</code>	Wait for background compactions to finish
<code>flush</code>	Trigger manual flush